

Containerizing HPC Workflows

A practical introduction to Apptainer

Junaid Mir • 22 September 2026 • SciCoCo.nrw 2026, RWTH



Outline

- Why Containers in HPC
- Container Fundamentals
- Interacting with a Container
- Building Your Own Container
- Running Containers on HPC with Slurm
- NVIDIA NGC Containers
- Advanced Topics
- Domain Examples
- Wrap-Up & Q&A

Session 1

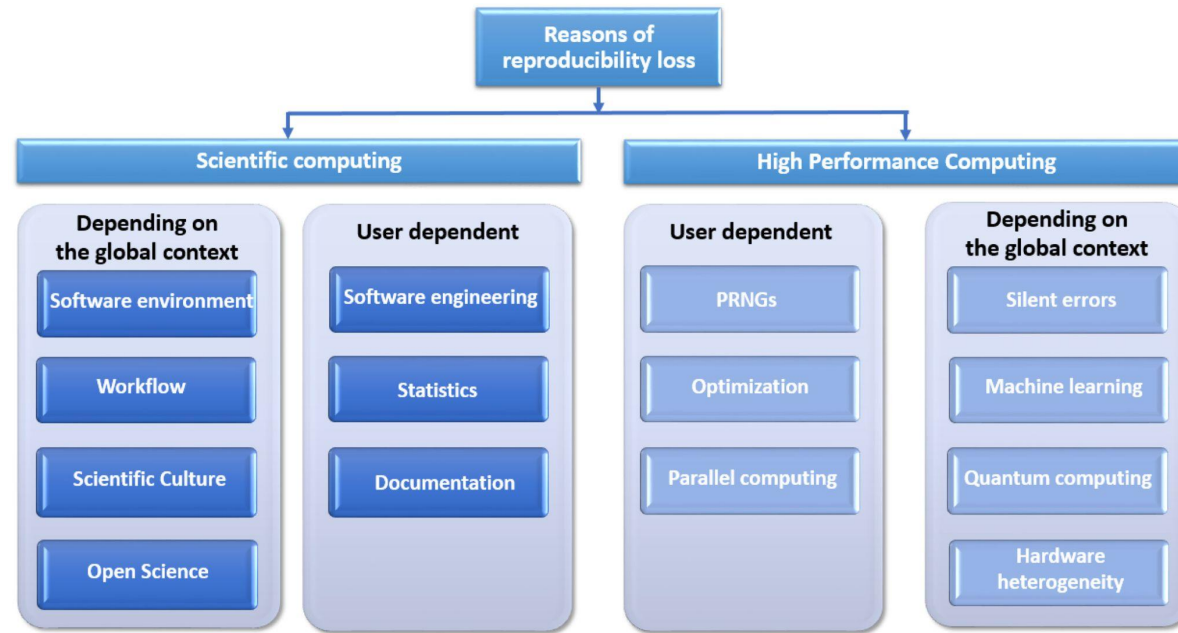
Foundations & Building

approx. 90 minutes

Why Containers in HPC

The Reproducibility Problem

- **“It works on my machine”** --- but not on the cluster
- Scientific software has deep dependency trees: compilers, MPI, CUDA, math libraries, Python/R stacks
- Version drift between development, testing, and production HPC environments
- Journals and funders increasingly require reproducible, shareable computational pipelines



Source: [Reproducibility, Replicability and Repeatability](#)

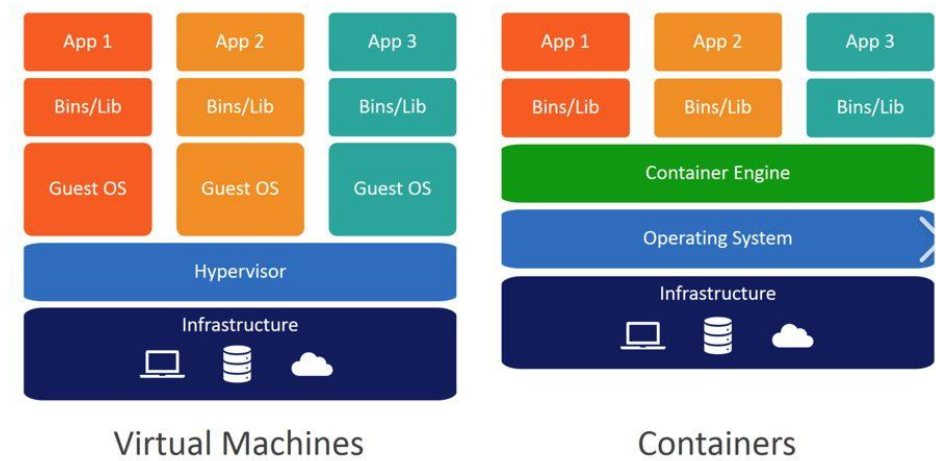
From Bare Metal to HPC Containers

- **Bare metal era** --- one application per server, low utilization
- **VM era** --- hypervisors (e.g. VMware) enable safe consolidation, but with OS overhead
- **Container era** --- lightweight, fast, portable workloads sharing one kernel
- **Docker** --- makes Linux containers usable for everyone
- **HPC containers** --- Apptainer adapts containers to secure, multi-user clusters

Bare metal	VMs	Linux containers	Docker	HPC containers
Physical servers	Consolidation	Kernel features	Image workflows	Secure multi-user HPC

Each step reduced waste and increased portability --- containers are the latest step, not the first.

Containers vs. Virtual Machines



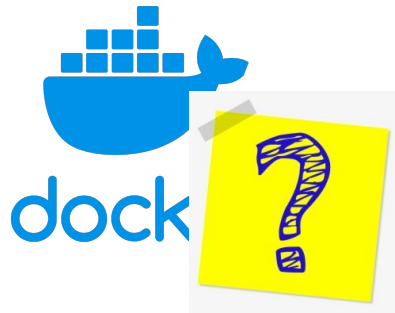
Source: [BMC Software \(2024\)](#)

	Virtual Machines	Containers
Kernel	Own guest kernel per VM	Shares host kernel
Startup time	Seconds to minutes	Milliseconds
Isolation	Strong (hardware-level)	Process-level (namespaces/cgroups)
Image size	GBs (full OS)	MBs–GBs (app + deps only)
Density per host	Low (tens)	High (hundreds+)
Typical HPC use	Rare --- overhead too high	Common --- near-native performance

Why Docker Is Not Suitable for HPC Systems

- **Docker's strengths**

- Mature ecosystem, huge image library
- Great for DevOps / cloud-native microservices
- Rich tooling and integrations



- **But problematic for HPC**

- **Rootful** Docker (**dockerd**) gives the user real root --- a non-starter on shared clusters
- **Rootless** Docker exists, but isn't yet feature-complete --- networking is the main gap
- Not designed for multi-user batch systems
- Security model conflicts with shared HPC clusters

- Apptainer isn't the only option --- Podman, Charliecloud, Shifter, and Enroot/Pyxis are also used on HPC systems
- *Docker is powerful for cloud and DevOps, but unsafe for HPC clusters --- this is why Apptainer was created for scientific computing.*

A Broader Lens: OCI-Native HPC Runtimes

- **OCI-native, daemonless alternatives**

- **Podman** --- OCI-native, daemonless and rootless by design; no separate image format needed
- **podman-hpc** (LANL, adopted by CSCS) --- adds bind-mounts, a SquashFS storage driver, and Slurm integration on top of standard Podman
- **Sarus** (ETH/CSCS) --- an earlier daemonless, OCI-native HPC runtime; CSCS's move to podman-hpc continues that lineage
- Stays inside standard Docker-format registries and tooling end-to-end

- **Why Apptainer still leads at most sites**

- Still the widest install base across HPC centers worldwide
- SIF is a single, self-contained file --- easy to **scp** between clusters without a registry
- Works without user namespaces, which some HPC sites still restrict
- Years of production use with Slurm, GPUs, and MPI at scale

- *The real fault line isn't Docker vs. Apptainer --- it's daemon+root vs. daemonless/rootless, and Apptainer isn't the only answer to that.*

Singularity and Apptainer

- Originally: Singularity, created for scientific/HPC use cases
- 2021: project forked --- **Apptainer** and **SingularityCE**
- **Apptainer** joined the **Linux Foundation** --- community-governed, vendor-neutral
- **SingularityCE** continues under **Sylabs**
- A fork, not a hand-off: **both projects remain actively developed today**
- Command-line interface and image format remain almost identical
- Apptainer is now the de facto standard on most HPC centers (incl. many in Germany)



APPTAINER



Who Uses Containers?

- **Researchers & Scientists** --- reproducible experiments, shareable software environments
- **HPC Users & Supercomputing Centers** --- identical environments across nodes/systems, GPU and MPI workloads with optimized libraries
- **Data Scientists & AI/ML Practitioners** --- Ready-to-use CUDA/cuDNN/PyTorch/TensorFlow stacks, NVIDIA NGC GPU-optimized containers, faster onboarding
- **Industry & DevOps Teams** --- microservices, CI/CD pipelines, versioned and isolated service environments
- **Software Developers** --- consistent dev environments, easy sharing across teams, simplified dependency management
- *From scientists to DevOps --- containers deliver portability, reproducibility, and isolation.*



Source: <https://makeameme.org/meme/basically-everyone-man>

Container Fundamentals

What is a container?

- A container is a packaged, portable user space consisting of an application, a root file system, and its environment
 - Allows "swapping" in and out of OS-es and environments (think lightweight VMs)
 - Shares the kernel with the host for high performance
 - Utilizes Linux namespaces
- In a nutshell, containers are encapsulation of system environments
- Ubiquitous in modern computing



**THESE GUYS GET YOUR
STUFF WHERE IT NEEDS TO GO**

[source](#)

Under the Hood: Namespaces & Cgroups

- **Namespaces:** isolate what a process can *see* (PIDs, mounts, network, users, ...)
- **Cgroups:** limit what a process can *use* (CPU, memory, I/O)
- A container is just a regular Linux process with a restricted view of the system
- No hypervisor, no separate kernel --- this is why it's fast

Images vs. Containers

- **Image**: a static filesystem snapshot (the blueprint)
- **Container**: a running instance of that image (the process)
- Apptainer/Singularity images are single files: the **SIF** format (Singularity Image Format)
- **SIF image is read-only *by default***, not by necessity --- **--writable** (a writable sandbox) or **--fakeroot** lets you modify it
- SIF is self-contained, cryptographically signable, and easy to copy between systems

Docker vs. Singularity/Apptainer

	Docker	Singularity / Apptainer
Daemon required	Yes (root)	No
Runs as	root	user
Image format	layers (OCI)	SIF
HPC/Slurm friendly	Limited	Yes
Typical use	Cloud/microservices	HPC

What makes Apptainer different?

- **Simple security model**

- UID/GID context is the same inside and outside the container
- Privilege escalation prevented via well-known kernel and file system features
- No daemon (reduced attack surface)

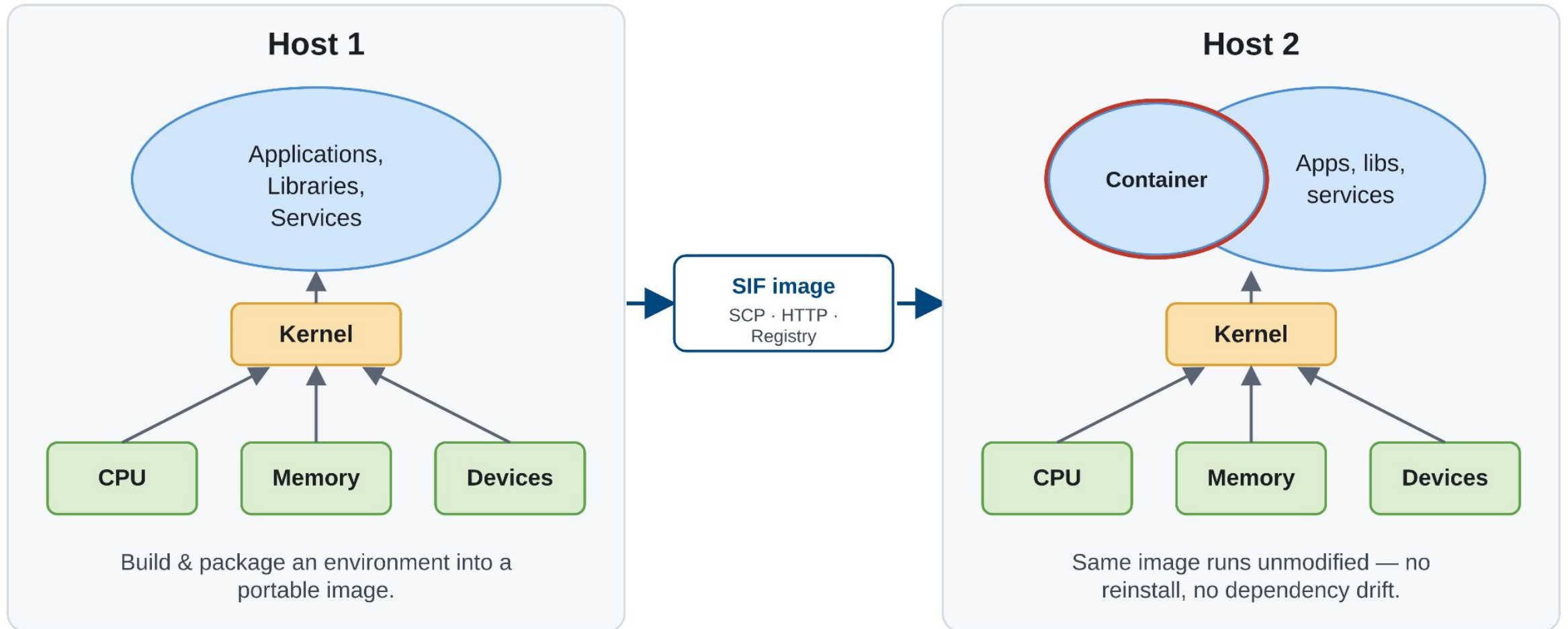
- **Integration over isolation**

- Simplifies host file operations
- GPUs, high-speed interconnects, MPI
- Minimal namespace isolation

- **Singularity Image Format (SIF)**

- Extremely portable and easy to manage
- Immutable container format
- Cryptographic signatures
- In-memory decryption

Container Portability: Build Once, Run Anywhere



Interacting with a Container

Before You Start: Where to Work

Home quota is 5 GB. For hands-on: pulling, building, running images --- happens in your personal project folder, not home

```
export WORK=$PROJECT_training2638/container-workshop/$USER
echo "export WORK=$PROJECT_training2638/container-workshop/\$USER" >> ~/.bashrc

mkdir -p $WORK/apptainer_cache $WORK/apptainer_tmp

rm -rf $HOME/.apptainer          # safe if you haven't pulled/built anything yet
ln -s $WORK/apptainer_cache $HOME/.apptainer

export APPTAINER_TMPDIR=$WORK/apptainer_tmp
echo 'export APPTAINER_TMPDIR=$WORK/apptainer_tmp' >> ~/.bashrc
```

- **Run these now**, before your first **pull** or **build**
- Add both **export** lines to **~/.bashrc** too --- otherwise they're gone in a new terminal
- **\$WORK** is your personal work area --- build and store your **.sif** files there
- **01_interacting/**, **02_building/**, **03_running_on_hpc/** are shared, read-only reference material
- **apptainer_tmp/** must exist before you **build** --- re-run **mkdir** if it's ever missing

Pulling an Existing Image

```
# From Docker Hub  
apptainer pull docker://ubuntu:22.04
```

```
# From the Sylabs Cloud Library  
apptainer pull library://library/default/alpine:latest
```

- Produces a local **.sif** file
- OCI/Docker images are converted automatically to SIF

shell, exec, run

```
apptainer shell ubuntu_22.04.sif      # interactive shell

apptainer exec ubuntu_22.04.sif cat /etc/os-release

apptainer run ubuntu_22.04.sif        # runs the %runscript
```

- **shell** --- opens an interactive prompt *inside* the container
- **exec** **<cmd>** --- runs a single command and returns to the host
- **run** --- triggers the image's **%runscript**, its predefined default action
- Notice: your home directory is bind-mounted automatically
- **The container filesystem is read-only by default** --- files inside it cannot be edited directly unless you add **--writable** (a writable image/sandbox) or **--fakeroot** (simulated root for build-time changes)
- **Exercise 1:** pull an image and explore its filesystem --- **apptainer exec <image> whoami** should print *your* username, not root

Bind Mounts & Environment

What is a bind mount? Maps a directory on the *host* into the container --- used to access input data, code, or output directories. Common paths (**\$HOME**, current directory) are often bound automatically.

```
apptainer exec --bind /scratch:/data ubuntu_22.04.sif ls /data
```

```
apptainer exec --env MYVAR=42 ubuntu_22.04.sif env | grep MYVAR
```

- **--bind** maps host paths into the container; **--env** passes environment variables in
- **Syntax matters:** **--bind /host/path:/container/path** --- host path first, container path second, separated by a colon
- Host **\$HOME**, **/tmp**, and current dir are bound by default (site-configurable)
- **Why:** keep data on HPC filesystems (home, scratch, project) --- avoid baking large datasets into the image

Building Your Own Container

Fakeroot in Apptainer

What is `fakeroot`? Simulates root *inside* the container, without real root on the host. Lets you install packages or modify the system image as if you were root --- useful for building and testing containers as a normal user.

```
apptainer build --fakeroot image.sif image.def
```

```
apptainer shell --fakeroot image.sif
```

- **Typical use cases:** installing software during a build (%**post**); running build steps that expect root privileges

Iterative Development with a Sandbox

What is a sandbox? A writable directory-based container --- build it quickly, enter it, install or tweak things, then freeze the result into an immutable, shareable **.sif** image.

```
apptainer build --sandbox sandbox/ docker://ubuntu:22.04
```

```
apptainer shell --fakeroot --writable sandbox/
```

```
apptainer build final.sif sandbox/
```

- **--sandbox** produces a writable directory instead of a compressed, read-only **.sif**
- **--writable** lets you edit files and install packages inside it directly --- combine with **--fakeroot** for real package installs
- **Iterate fast:** edit, test, repeat --- no full rebuild-from-scratch cycle while developing
- **Once it works,** **apptainer build final.sif sandbox/** freezes it into the immutable, shareable image
- The **.def** file stays the reproducible record --- treat the sandbox as scratch space, not the source of truth

Anatomy of a Definition File

```
Bootstrap: docker
From: ubuntu:22.04
%post
    apt-get update && apt-get install -y bash
%environment
    export GREETING="Hello from Apptainer!"
%runscript
    echo $GREETING
    echo "This container was built on $(date)"
    exec "$@"
%labels
    Author Student
    Version v1.0
```

- **Bootstrap/From:** base image source (docker, library, yum, debootstrap, ...)
- **%post:** commands run once at build time (installs, compilation)
- **%environment:** variables set every time the container runs
- **%runscript:** default action for **apptainer run**
- **%labels:** metadata --- author, version, custom key/value pairs

Building the Image

```
# Local build (requires root or --fakeroot)
apptainer build --fakeroot hello.sif exercises/01_hello.def
```

- `--fakeroot` emulates root privileges for unprivileged users
- **Exercise 2:** build `exercises/01_hello.def`, then `apptainer run hello.sif` --- do you see the greeting?

Beyond the Basics: More Definition-File Sections

```
%test
python -c "import numpy; print('Numpy version:', numpy.__version__)"
```

- **%setup**: runs on the *host* before the OS is installed --- prepares files to copy in
- **%files**: copies files from the host into the container
- **%startscript**: runs when a container *instance* is started (service-like)
- **%help**: text shown by `apptainer run-help`
- **%test**: a self-check run by `apptainer test` --- the idiomatic way to verify a build
- **Exercise 3**: build `exercises/02_conda.def`, then `apptainer test conda.sif` --- confirms the **%test** block passes
- **Why this matters**: **%post** resolves and locks your conda environment once, *at build time* --- the image ships a fully pre-built, version-locked environment
- **Exercise 4**: build `exercises/03_complex.def` (all five sections combined), then try `apptainer instance start/list/stop`

Apptainer Instances

What is an instance? A named, background container that keeps running after your command returns --- unlike `shell`, `exec`, or `run`, which exit immediately.

```
apptainer instance start app.sif myinstance
```

```
apptainer exec instance://myinstance ps aux
```

```
apptainer instance stop myinstance
```

- `apptainer instance list` shows all running instances for your user
- Runs the image's `%startscript` on start and `%stopscript` on stop
- **Use cases:** long-running services --- a Jupyter server, a database, a web app inside the container
- Each instance gets its own private namespace but stays reachable via `exec instance://name`
- **Clean up when done:** an instance keeps running (and holding resources) until you stop it
- **Try it:** start an instance from `complex.sif` (built in Module 2), run `ps aux` against it, then stop it --- no new files needed

Inspecting a Built Image

What does `apptainer inspect show`? Prints an image's embedded metadata --- labels, environment variables, and even the original definition file --- without needing to run it.

```
apptainer inspect image.sif
```

```
apptainer inspect --deffile image.sif
```

```
apptainer inspect -r image.sif
```

- **Useful for auditing images you didn't build yourself:** what base OS, what labels, what environment?
- `--deffile` recovers the original `.def` file if it was embedded at build time and you've lost your copy
- `-r` prints only the `%runscript`; `-e` prints only environment variables; `-t` requests JSON output for scripting
- Works on any `.sif`, local or freshly pulled --- no need to run it, or even have permission to execute it
- **Try it:** run `apptainer inspect` and `--deffile` against any image you already built --- `hello.sif`, `conda.sif`, or `complex.sif`

Signing & Verifying Images

Why sign a container image? On shared HPC storage, anyone with read access could tamper with a `.sif` file. A cryptographic signature lets you detect that before you trust and run it.

```
apptainer key newpair
```

```
apptainer sign image.sif
```

```
apptainer verify image.sif
```

- **Signatures are embedded directly in the SIF file:** no separate keyserver or PKI infrastructure required
- `apptainer key newpair` generates a personal PGP keypair (or `apptainer key import` for an existing one)
- `apptainer verify` checks the signature and warns if the image has changed since it was signed
- **Since Apptainer 3.11**, X.509 certificates are supported alongside the default PGP signing
- **Best practice:** sign images you build yourself before sharing them; verify images you pull from others
- **Try it:** generate a keypair, then sign and verify an image you already built --- no new files needed

From Dockerfile to Definition File

Dockerfile

```
FROM ubuntu:22.04
RUN apt-get update && \
    apt-get install -y python3
ENV APP_ENV=production
COPY app.py /app/app.py
ENTRYPOINT ["python3", "/app/app.py"]
```

Definition file

```
Bootstrap: docker
From: ubuntu:22.04
%post
    apt-get update && \
        apt-get install -y python3
%environment
    export APP_ENV=production
%files
    app.py /app/app.py
%runscript
    python3 /app/app.py
```

FROM→**Bootstrap/From** **RUN**→**%post** **ENV**→**%environment** **COPY**→**%files** **ENTRYPOINT**→**%runscript**

Tip: you don't always need to translate line by line --- point `Bootstrap: docker` straight at an existing Dockerfile-built

```
Bootstrap: docker
From: myregistry.io/myproject/myimage:1.0
```

Session 2

HPC Integration & Capstone

approx. 90 minutes

Running Containers on HPC with Slurm

Apptainer on Cluster Systems

Key idea: Slurm handles *resources*, Apptainer handles the *environment*.

- **Slurm's job**

- Allocate CPUs, memory, GPUs, nodes
- Schedule jobs in queues/partitions
- Launch tasks via `srun/sbatch`



- **Apptainer's job**

- Provide a reproducible software stack (the `.sif` image)
- Run the application inside the container
- Preserve your user ID and file permissions --- true for the default *rootless* (sandbox) mode; sites that configure `sudo`-based Singularity can behave differently

- **In practice**

- Usually available as a module: `module load apptainer`
- No daemon, no root --- runs fine under the scheduler's user context
- Site admins may configure default bind paths (e.g. `/scratch`, `/work`)
- Some sites also bind-mount the *host's* MPI libraries into the container by default, for full-speed communication --- usually works, but ABI or version mismatches can occur and are confusing to debug

A Minimal sbatch Script

```
#!/bin/bash
#SBATCH --job-name=matmul-cpu-scratch
#SBATCH --account=training2638
#SBATCH --partition=dc-cpu
#SBATCH --nodes=1
#SBATCH --cpus-per-task=16
#SBATCH --time=00:10:00
#SBATCH --output=matmul_%j.log
apptainer exec ../02_building/conda.sif python3 matmul_cpu.py
```

- **#SBATCH** lines **allocate hardware** --- nodes, CPUs, time --- before anything runs
- **apptainer exec** then **executes inside** that already-granted allocation --- Slurm hands out the resources, Apptainer just uses them
- Single-node (above): the script body runs once, directly on the allocated node
- Multi-node: prefix with **srun** --- e.g. **srun apptainer exec mytool.sif ...** --- to launch one container instance per task across all allocated nodes

Filesystems and Resource Requests

- **HPC filesystems:** `$HOME` (small files, scripts) • Scratch/Work (large I/O, temp data) • Project/Group storage (shared datasets)
- Always bind the parallel/shared filesystem explicitly --- don't assume it's mounted
- Bind only the directories you need, e.g. `--bind $SCRATCH/data:/data`
- Match `--cpus-per-task` in Slurm to threads used inside the container
- **Exercise 5:** submit the sbatch script `job_cpu_matmul.slurm` inside `(../03_running_on_hpc)` and check the log
- *Rule of thumb: data stays on the HPC filesystem --- software stays in the container.*

NVIDIA NGC Containers

What Is NGC?

- NVIDIA GPU Cloud catalog: pre-built, GPU-optimized containers
- Covers deep learning (PyTorch, TensorFlow), HPC (CUDA, NAMD, GROMACS), and data science
- Tuned and validated by NVIDIA --- saves weeks of dependency wrangling
- **Especially relevant for AI workloads:** the standard starting point for LLM training, fine-tuning, and inference --- CUDA, cuDNN, and NCCL versions are pre-matched, which is exactly what's easy to get wrong building a GPU stack by hand
- This is the same PyTorch base image our LLM fine-tuning capstone builds on later today



Do You Need an NGC Account to Pull Images?

- **Short answer:** it depends on the container.
- **You can pull without an account when:** the NGC container is publicly available; NVIDIA marks it as “Guest accessible”; most AI/ML + CUDA base images fall into this category
- **You must sign in when:** the container is enterprise, partner, or licensed; the registry returns **unauthorized: authentication required**; you need an API key to access locked content
- *Practical rule: try to pull the image --- if it fails, you need an NGC account + API key.*

Pulling and Running an NGC Container

```
apptainer pull docker://nvcr.io/nvidia/pytorch:24.01-py3

apptainer exec --nv ngc_pytorch.sif \
    python3 -c "import torch; print(torch.cuda.is_available())"
```

- `--nv` injects the host's NVIDIA driver and GPU devices into the container
- **Driver compatibility:** `--nv` passes the *host* driver in --- the CUDA toolkit ships *inside* the image, so the host driver version must meet or exceed what that toolkit requires
- Mismatch shows up as a runtime error, not a pull/build failure --- check `nvidia-smi` on the node if a GPU check fails
- **Exercise 6:** run the GPU check above on a GPU node
- **Exercise 7:** build `exercises/04_ml_gpu.def` (PyTorch + scikit-learn) and run its `%test`

Advanced Topics

MPI-Based Applications

- **Hybrid model:** MPI launcher on the host, MPI library inside the container --- versions must be ABI-compatible
 - `mpirun/srun` runs on the *host*, **outside** the container --- it's the host resource manager, not Apptainer, that starts one process per rank
 - That host launcher then spawns `apptainer exec` once per rank, across every allocated node --- each rank runs *inside its own container instance*
 - `srun apptainer exec mpi_app.sif ./my_mpi_program`
- Mismatched MPI versions are the #1 cause of container-MPI failures

GPU-Enabled Applications

- `--nv` for NVIDIA, `--rocm` for AMD GPUs
- Host driver is used; container only needs the CUDA/ROCm *toolkit*, not the driver
- Driver/toolkit version compatibility still matters --- check NVIDIA's compatibility matrix

Optimizing I/O-Intensive Workflows

- Avoid heavy metadata operations (many small files) directly on parallel filesystems
- Use local NVMe/scratch inside the job, stage data in/out
- **Pull once, not per node:** pull/build the `.sif` a single time onto shared storage (home, project, scratch) --- don't let every compute node in a multi-node job pull from Docker Hub simultaneously, that's a self-inflicted I/O and network bottleneck (and registries may rate-limit you)

Domain Examples

Capstone: An LLM Fine-Tuning Container

```
Bootstrap: docker
From: nvcr.io/nvidia/pytorch:23.08-py3    # NGC base: CUDA, cuDNN, PyTorch

%post
    apt-get update && apt-get install -y git
    pip install transformers datasets accelerate bitsandbytes peft

%test
    python -c "import transformers; print(transformers.__version__)"
```

- Real recipe: `exercises/05_llm.def` --- start from NGC, layer on fine-tuning libraries
- **Exercise 8:** build it, then run `exercises/llm_training_script.py` with `--nv`

The HuggingFace Cache Gotcha

```
mkdir -p /scratch/$USER/hf_cache  
apptainer exec --nv \  
    --bind /scratch/$USER/hf_cache:/root/.cache/huggingface \  
    llm_container.sif python exercises/llm_training_script.py
```

- Models (Llama, Mistral, ...) download to `~/.cache/huggingface` by default
- On HPC, `$HOME` is often quota-limited --- large models fill it fast
- **Fix:** bind-mount the cache to scratch, same pattern as any other bind path
- Ties together everything so far: GPU (`--nv`), bind paths, and I/O placement

Self-Assessment: What You Can Now Do

This drives where we spend Q&A:

- ☐ Explain why containers matter for HPC reproducibility
- ☐ Pull, run, and inspect an existing container image
- ☐ Write a definition file and build your own image
- ☐ Submit a containerized job via Slurm
- ☐ Run a GPU-enabled container with --nv
- ☐ Diagnose a common MPI/GPU/bind-mount failure

Wrap-Up & Q&A

Recap

- Containers help with reproducibility & portability without needing root --- not a silver bullet: portability still assumes the same CPU architecture (an ARM image simply won't run on an x86 node)
- Definition files are the recipe; SIF images are the reproducible artifact
- Apptainer integrates cleanly with Slurm, GPUs, and MPI
- NGC containers save you from building complex ML/HPC stacks by hand

Common Pitfalls

- Forgetting to bind the filesystem your data actually lives on
- MPI/CUDA version mismatches between host and container
- Relying on `:latest` tags instead of pinned versions
- Building as root without `--fakeroot` on systems where you shouldn't

References

- Charlie Cloud charliecloud.io
- Apptainer docs: apptainer.org/docs
- Sylabs Cloud Library: cloud.sylabs.io
- NVIDIA NGC catalog: catalog.ngc.nvidia.com
- The Linux Foundation : <https://www.linuxfoundation.org/>
- All exercise files from today: github.com/mirjunaid26/hpc_containers

Thank you for your attention!

Contact

Junaid Mir
ZiM, Universität Duisburg-Essen
Schützenbahn 70, 45127 Essen

